DOP-C02 · 內容領域 1

SDLC 自動化

1.1 實作 CI/CD 管道 · 1.2 整合自動化測試
1.3 建置與管理產物 · 1.4 實作部署策略

22%

考試計分占比

依 AWS 官方考試指南（DOP-C02）重新整理

領域 1 總覽：四項任務陳述

1.1

實作 CI/CD 管道

SDLC 概念與模型 · 單一 / 多帳戶管道模式 · 程式碼 / 映像 / 產物儲存庫 · CodeBuild · 機密管理 · 部署策略

1.2

將自動化測試整合到 CI/CD 管道

測試類型與放置階段 · PR / 合併觸發測試 · 負載與效能測試 · 以退出碼判斷健康 · 單元測試與覆蓋率

1.3

建置與管理產物

產物使用案例與安全管理 · CodeArtifact / S3 / ECR · 以 CodeBuild、Lambda 產生產物 · EC2 Image Builder

1.4

為執行個體、容器與無伺服器環境實作部署策略

EC2 / ECS / EKS / Lambda · EFS / S3 / EBS · 可變 vs 不可變 · CodeDeploy 代理程式 · 疑難排解 · 藍綠 / 金絲雀

SDLC 的階段與開發模型



瀑布式 Waterfall

階段依序進行，前一階段完成才進入下一階段。需求穩定時可行，但回饋慢、後期變更成本高

敏捷 Agile

以短迭代交付可運作的增量，持續取得使用者回饋並調整優先順序

DevOps

開發與維運協作，以 CI/CD 自動化、基礎設施即程式碼與監控縮短交付週期

持續整合 CI

開發者頻繁合併到主幹，每次提交自動建置並執行測試

持續交付 CD

每個通過測試的變更都可隨時發布；上線前保留人工核准

持續部署 CD

通過所有自動化檢查後直接部署到生產環境，無人工關卡

AWS 上的 CI/CD 管道組成

AWS CodePipeline　編排各階段（Stage）與動作（Action），以產物（Artifact）在階段間傳遞，產物存放於 S3 產物儲存區



來源 Source

- CodeCommit
- GitHub / GitLab / Bitbucket（CodeConnections）
- Amazon S3
- Amazon ECR





建置 Build

- AWS CodeBuild
- Jenkins（自訂動作）
- 產出建置產物與測試報告





測試 Test

- CodeBuild 測試專案
- Lambda / Step Functions 呼叫
- AWS Device Farm





核准 Approval

- 手動核准動作
- 搭配 SNS 通知審核人
- 持續交付的關卡





部署 Deploy

- AWS CodeDeploy
- Amazon ECS
- CloudFormation / Elastic Beanstalk
- Amazon S3



版本控制整合：以分支、Git 標籤或 PR 事件觸發管道（V2 管道支援分支 / 標籤 / 檔案路徑篩選）；不同分支對應不同環境，例如 develop → 測試、main → 生產

單一帳戶 vs 多帳戶的管道部署模式

單一帳戶

- 管道與 dev / test / prod 資源位於同一個帳戶
- 以不同階段或不同堆疊名稱區分環境
- 設定簡單、權限容易管理
- 隔離與權限邊界弱，誤操作可能影響生產
- 適合小型團隊、PoC、非生產工作負載

多帳戶（建議做法）

- 工具帳戶（Tooling）集中執行管道
- 每個目標帳戶建立跨帳戶 IAM 角色，供管道 assume
- 產物 S3 儲存貯體使用客戶管理 KMS 金鑰加密，並以金鑰政策 + 儲存貯體政策授權目標帳戶
- 以 AWS Organizations、CloudFormation StackSets 統一佈建
- 爆炸半徑小、符合稽核與合規要求

工具帳戶

CodePipeline · 產物 S3 · KMS

→

開發帳戶 Dev

跨帳戶部署角色

→

測試帳戶 Test

跨帳戶部署角色

→

生產帳戶 Prod

手動核准後部署

管理建置與部署機密

比較項目	Secrets Manager	Parameter Store（SecureString）
自動輪換	內建，透過 Lambda 輪換函數	無內建輪換
費用	每個密碼按月計費 + API 呼叫	標準參數免費；進階參數收費
大小上限	64 KB	標準 4 KB；進階 8 KB
跨帳戶共用	資源型政策	進階參數可透過 AWS RAM 共用
典型用途	資料庫密碼、需輪換的 API 金鑰	組態值、功能旗標、AMI ID

buildspec.yml

```
env:
  parameter-store:
    DB_HOST: /app/prod/db_host
  secrets-manager:
    DB_PASS: prod/db:password
phases:
  build:
    commands:
      - ./deploy.sh
```



- 不要把機密寫死在原始碼、buildspec 或環境變數明文中
- CodeBuild 服務角色需 secretsmanager:GetSecretValue / ssm:GetParameters，以及對應 KMS 金鑰的 kms:Decrypt
- CodeBuild 會在日誌中遮蔽來自 Secrets Manager / Parameter Store 的值

設定建置流程：CodeBuild 與 buildspec

buildspec 的執行階段

- **install** — 安裝執行環境與相依套件
- **pre_build** — 登入 ECR、下載相依、靜態檢查
- **build** — 編譯、執行單元測試、建置映像
- **post_build** — 推送映像、打包、發送通知

其他區段　env（變數與機密）· reports（測試 / 覆蓋率報告）· artifacts（輸出產物）· cache（快取路徑）

建置環境映像

AWS 受管映像，或自訂映像（可存放在 ECR）

運算類型

依 vCPU / 記憶體選擇；也可用 Lambda 運算加速短建置

VPC 存取

在 VPC 內建置以存取私有資源（RDS、內部儲存庫）

快取

本機快取或 S3 快取，減少重複下載相依套件

批次建置

建置矩陣 / 圖形，一次平行建置多種組合

日誌與通知

CloudWatch Logs / S3；建置狀態事件送 EventBridge

選擇部署策略：CodeDeploy 部署組態

運算平台	部署類型	預先定義的部署組態	適用情境
EC2 / 本地伺服器	就地 In-place 藍綠 Blue/green	OneAtATime · HalfAtATime · AllAtOnce	既有伺服器群就地更新；或替換為新的執行個體
AWS Lambda	流量轉移（別名）	Canary10Percent5Minutes 等 Linear10PercentEvery1Minute 等 AllAtOnce	以別名逐步把流量導向新版本
Amazon ECS	藍綠（兩組目標群組）	ECSCanary10Percent5Minutes 等 ECSLinear10PercentEvery1Minutes 等 ECSAllAtOnce	以 ALB / NLB 在原始與替換任務集間切換流量



停機容忍度

不能停機 → 藍綠 / 金絲雀



回滾速度

需秒級回滾 → 藍綠（切回原環境）



成本

資源有限 → 就地或滾動



風險控管

想先以少量流量驗證 → 金絲雀 / 線性

測試的類型



單元測試 Unit

驗證單一函數或類別的邏輯；最快、最便宜，每次提交都執行

CodeBuild + JUnit / pytest



整合測試 Integration

驗證多個元件或與外部服務（資料庫、API）間的互動

部署到測試環境後由 CodeBuild 執行



驗收測試 Acceptance

以業務需求確認功能符合預期，常作為上線前關卡

Staging 環境 + 手動核准



使用者介面測試 UI

模擬使用者操作瀏覽器或行動裝置，驗證畫面與流程

AWS Device Farm、Selenium



安全掃描 Security

SAST 原始碼、相依套件、容器映像、DAST 執行時期掃描

Amazon Inspector（ECR、Lambda）



負載 / 壓力測試

在高流量下量測效能、找出瓶頸與系統極限

Distributed Load Testing on AWS

在管道的哪個階段放哪種測試

提交 / PR	建置	測試環境	預備 Staging	生產 Prod
• 靜態程式碼分析 • 單元測試 • 程式碼覆蓋率 • SAST 掃描	• 單元測試 • 相依套件掃描 • 容器映像掃描 • 產生測試報告	• 整合測試 • API 測試 • UI 測試	• 驗收測試 • 負載 / 壓力測試 • 效能基準 • DAST 掃描	• 冒煙測試 • 合成監控（CloudWatch Synthetics） • 警示觸發自動回滾

快速、便宜、隔離 → 越往右越慢、越貴、越接近真實環境 → **真實流量與規模**

原則：越早發現問題、修正成本越低（Shift-left）；任何階段失敗即停止管道，不把缺陷往下游推

在 PR 與合併時自動建置、以退出碼判斷健康



0 結束碼 0 = 成功

CodeBuild 階段成功，產物傳到下一個階段，管道繼續執行

- 測試指令應以結束碼回報結果，而不是只印出訊息
- CodeDeploy 勾點指令碼同理：非零結束碼 → 部署失敗

≠0 非零結束碼 = 失敗

建置失敗、管道停止；可透過 EventBridge + SNS 通知團隊

- 在 buildspec 中用 on-failure: ABORT 控制階段失敗行為
- 應用程式健康檢查也依結束碼判斷（如 ValidateService 勾點）

單元測試、覆蓋率與大規模測試

CodeBuild 報告群組

- 測試報告：JUnit XML、Cucumber JSON、TestNG、NUnit、Visual Studio TRX
- 覆蓋率報告：JaCoCo、Cobertura、Clover、SimpleCov、LCOV
- 在 buildspec 的 reports 區段設定
- 可設定門檻，低於門檻就讓建置失敗

在管道中呼叫 AWS 服務測試

- Lambda 呼叫動作：執行自訂驗證並回報成功 / 失敗
- Step Functions 呼叫動作：編排多步驟測試
- CodeBuild 對已部署端點執行 API 測試
- Device Farm：在真實裝置上跑 UI 測試

大規模負載與效能測試

- Distributed Load Testing on AWS：以 Fargate 容器產生大量並行流量
- 建立效能基準，與前一版本比較回應時間與錯誤率
- 搭配 CloudWatch 指標觀察瓶頸
- 在接近生產規模的環境執行

產物的產生、生命週期與安全管理

產生

CodeBuild artifacts
Lambda 打包
SAM / CDK

版本化

語意化版本
不可變標籤
Git SHA

掃描與簽章

Inspector 掃描
AWS Signer 簽章

推廣

同一份產物
dev → test → prod

保留與清除

保留可回滾版本
過期自動清除

安全管理

- 靜態加密：S3 / ECR / CodeArtifact 皆可用 KMS
- 最小權限：IAM 政策 + 資源型政策（儲存庫政策）
- 完整性：AWS Signer 為 Lambda 程式碼簽章、容器映像簽章
- ECR 標籤不可變，避免同一標籤被覆寫

生命週期考量

- ECR 生命週期政策：依映像數量或天數清除
- S3 版本控制 + 生命週期規則：轉 Glacier 或過期
- CodeArtifact 套件版本狀態：Published / Unlisted / Archived / Disposed
- Build once, deploy many：各環境部署同一份產物

產物儲存庫：CodeArtifact、S3、ECR



AWS CodeArtifact

軟體套件

- 支援 npm、PyPI、Maven、NuGet 等格式
- 網域（Domain）集中管理多個儲存庫
- 上游儲存庫與外部連線代理公開來源
- aws codeartifact login 取得授權權杖（最長 12 小時）



Amazon S3

通用產物

- CodePipeline 產物儲存區
- CodeDeploy 修訂版本（zip / tar）
- CloudFormation 範本、Lambda 套件
- 版本控制、KMS 加密、跨區域複寫



Amazon ECR

容器映像

- 私有與公有儲存庫，支援 OCI 產物
- 映像掃描：基本或增強（Amazon Inspector）
- 跨區域 / 跨帳戶複寫、提取快取
- 標籤不可變、生命週期政策

以 EC2 Image Builder 自動建置 AMI 與容器映像

來源映像

AWS 受管 AMI 或既有映像

建置元件

安裝套件、強化設定（YAML 元件文件）

測試元件

啟動執行個體驗證映像

分發

AMI 跨區域 / 跨帳戶；容器推到 ECR

映像配方 / 容器配方　定義來源映像 + 要套用的元件

基礎架構組態　執行個體類型、IAM 執行個體設定檔、VPC

分發設定　目標區域、帳戶、啟動範本、ECR 儲存庫

排程與觸發　cron 排程，或在基礎映像更新時自動重建

弱點掃描　整合 Amazon Inspector 掃描產出映像

生命週期政策　自動停用、刪除過舊 AMI 與快照

黃金映像 + 不可變基礎設施：修補作業系統不登入伺服器，而是由管道產出新 AMI，再以 Auto Scaling 執行個體重新整理或藍綠部署替換舊執行個體

各平台的部署方法

平台	常見部署方式	工具與服務	重點
Amazon EC2 / 本地伺服器	就地更新、藍綠（替換執行個體或 Auto Scaling 群組）	CodeDeploy（需代理程式）、Elastic Beanstalk、Auto Scaling 執行個體重新整理	appspec.yml 與生命週期勾點
Amazon ECS	滾動更新、藍綠、金絲雀 / 線性	ECS 部署控制器、CodeDeploy	部署斷路器可自動回滾；藍綠需兩組目標群組
Amazon EKS	滾動更新；藍綠 / 金絲雀需額外工具	kubectl、Helm、GitOps（Argo CD、Flux）、Argo Rollouts	就緒探針、PodDisruptionBudget 確保可用性
AWS Lambda	別名流量轉移：金絲雀、線性、一次全部	CodeDeploy、AWS SAM（DeploymentPreference）	版本 + 別名；CloudWatch 警示觸發自動回滾

可變 vs 不可變部署模式

可變 Mutable

- 在既有伺服器上就地更新程式與設定
- 速度快、不需額外資源
- 容易產生組態漂移（configuration drift）
- 回滾需重新部署舊版本，較慢
- 例：CodeDeploy 就地部署、Beanstalk All at once / Rolling

不可變 Immutable

- 建立全新資源，驗證後再取代舊資源
- 環境一致、可預測，無組態漂移
- 回滾快：切回仍存在的舊資源
- 暫時需要雙倍資源與較長部署時間
- 例：新 AMI + 新 ASG、Beanstalk Immutable、藍綠、容器

Elastic Beanstalk 部署政策：由可變到不可變

All at once	Rolling	Rolling with additional batch	Immutable	Traffic splitting	Blue/green
最快、會停機	分批、容量下降	先加一批維持容量	新 ASG 驗證後替換	金絲雀式流量分割	新環境 + 交換 URL

部署方法比較：藍綠、金絲雀、線性、滾動



藍綠 Blue/green

建立完整的新環境（綠），驗證後把流量一次切換過去；藍環境保留以便回滾

回滾：最快

成本：雙倍資源



金絲雀 Canary

先把小比例流量（如 10%）導向新版本，觀察一段時間無異常後再全部切換

回滾：快

風險：最低



線性 Linear

以固定間隔、固定比例逐步增加流量（如每分鐘 10%），直到 100%

回滾：快

部署時間：較長



滾動 Rolling

分批更新執行個體或任務，每批完成健康檢查後再處理下一批

回滾：中等

容量：暫時下降



一次全部 All at once

同時更新所有目標，最快但所有使用者同時受到影響

回滾：慢

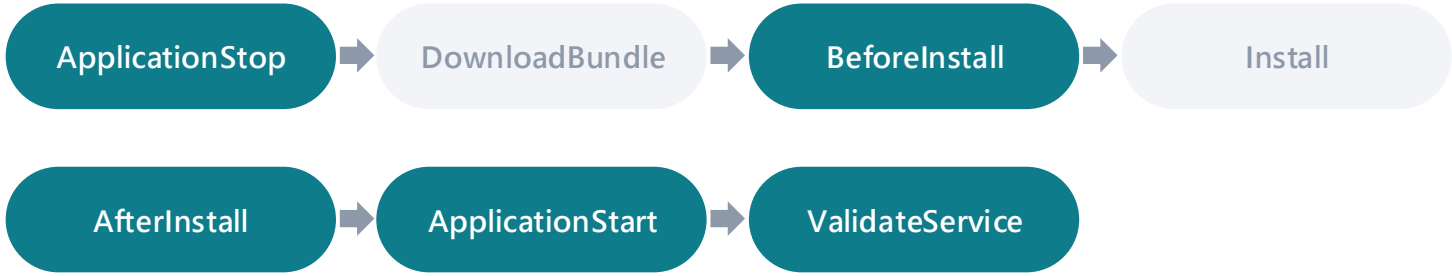
風險：最高

CodeDeploy 代理程式與 AppSpec 生命週期勾點

CodeDeploy 代理程式

- 僅 EC2 / 本地伺服器需要；Lambda、ECS 不需要
- 可用 Systems Manager（AWS-ConfigureAWSPackage）安裝與自動更新
- 代理程式輪詢 CodeDeploy 取得部署工作
- 執行個體設定檔需能讀取 S3 修訂版本
- 日誌：/var/log/aws/codedeploy-agent/

EC2 / 本地伺服器的勾點順序（就地部署）



■ 可放自訂指令碼　■ 由代理程式執行，不能放指令碼

有負載平衡器時

- BeforeBlockTraffic → BlockTraffic → AfterBlockTraffic（先從 LB 移除）
- BeforeAllowTraffic → AllowTraffic → AfterAllowTraffic（再加回 LB）

Lambda / ECS 的勾點

- Lambda：BeforeAllowTraffic / AfterAllowTraffic
- ECS：BeforeInstall / AfterInstall / AfterAllowTestTraffic / BeforeAllowTraffic / AfterAllowTraffic
- 勾點以 Lambda 函數實作驗證

應用程式儲存模式與產物存取權限



Amazon EFS

共享檔案系統，跨 AZ 由多台 EC2、ECS 任務、Lambda 同時掛載；適合共用內容與上傳檔案



Amazon S3

物件儲存；靜態資產、部署套件、使用者檔案。把狀態移出伺服器，執行個體才能被任意替換



Amazon EBS

區塊儲存，綁定單一 AZ 與單一執行個體；不可變部署時，資料需快照或移到外部儲存



存取產物儲存庫所需權限

EC2 執行個體設定檔

s3:GetObject 讀取 CodeDeploy 修訂版本

CodeArtifact

codeartifact:GetAuthorizationToken + sts:GetServiceBearerToken，再加讀取儲存庫權限

ECR（ECS 任務執行角色）

ecr:GetAuthorizationToken / BatchGetImage / GetDownloadUrlForLayer

跨帳戶

資源型政策（儲存貯體 / 儲存庫政策）+ KMS 金鑰政策

部署疑難排解

代理程式未執行或版本過舊

症狀　部署卡在 Pending、執行個體無回應

處理　檢查服務狀態、以 SSM 重新安裝或更新代理程式

執行個體設定檔權限不足

症狀　DownloadBundle 失敗、S3 Access Denied

處理　附加可讀取修訂版本 S3 與 KMS 的角色

AppSpec 錯誤

症狀　找不到 appspec.yml、路徑或格式錯誤

處理　確認位於套件根目錄、YAML 縮排與 files 路徑

勾點指令碼失敗

症狀　指令碼逾時或回傳非零結束碼

處理　查看代理程式日誌、調整 timeout、修正指令碼權限

健康檢查失敗

症狀　ALB 目標不健康、ValidateService 失敗

處理　確認健康檢查路徑、安全群組與應用程式啟動時間

ECS 任務無法啟動

症狀　映像拉取失敗、資源不足、斷路器觸發

處理　檢查任務執行角色、ECR 權限、CPU / 記憶體設定



自動回滾：部署失敗或 CloudWatch 警示觸發時回到上一個成功版本。排查工具：CodeDeploy 部署事件、代理程式日誌、CloudWatch Logs、CloudTrail

情境練習

1. 多帳戶管道的產物需讓目標帳戶解密

2. CodeBuild 用的資料庫密碼需定期自動輪換

3. 每次開 PR 都自動跑單元測試並回報

4. 測試指令回傳非零結束碼

5. 集中管理 npm / PyPI 套件並代理公開來源

6. 每月自動修補並產出黃金 AMI 到多區域

7. 自動清除 30 天以上未標記的容器映像

8. Lambda 新版本先導 10% 流量 5 分鐘

9. 零停機並可立即切回舊版本

10. CodeDeploy 部署卡住、EC2 沒有回應

情境練習

對照你的答案，再回頭複習對應任務

1. 多帳戶管道的產物需讓目標帳戶解密 — **客戶管理 KMS 金鑰**

2. CodeBuild 用的資料庫密碼需定期自動輪換 — **Secrets Manager**

3. 每次開 PR 都自動跑單元測試並回報 — **CodeBuild PR 觸發**

4. 測試指令回傳非零結束碼 — **建置失敗、管道停止**

5. 集中管理 npm / PyPI 套件並代理公開來源 — **CodeArtifact 上游**

6. 每月自動修補並產出黃金 AMI 到多區域 — **EC2 Image Builder**

7. 自動清除 30 天以上未標記的容器映像 — **ECR 生命週期政策**

8. Lambda 新版本先導 10% 流量 5 分鐘 — **Canary10Percent5Minutes**

9. 零停機並可立即切回舊版本 — **藍綠部署**

10. CodeDeploy 部署卡住、EC2 沒有回應 — **檢查代理程式與 IAM 角色**

自我檢核：領域 1 官方目標

1.1	**說明 SDLC 概念、階段與模型**	瀑布、敏捷、DevOps、CI / 持續交付 / 持續部署
1.1	**設計單一 / 多帳戶管道**	工具帳戶、跨帳戶角色、客戶管理 KMS 金鑰
1.1	**設定儲存庫、建置與機密**	CodeConnections、CodeBuild、Secrets Manager、Parameter Store
1.1	**選擇合適的部署策略**	CodeDeploy 部署組態、停機與回滾需求
1.2	**依階段安排測試類型**	單元、整合、驗收、UI、安全掃描、負載測試
1.2	**PR 觸發測試、以退出碼判斷**	Webhook / EventBridge、結束碼 0 與非 0
1.2	**自動化覆蓋率與大規模測試**	報告群組、Lambda 呼叫動作、Distributed Load Testing
1.3	**建立與保護產物儲存庫**	CodeArtifact、S3、ECR、KMS、生命週期政策
1.3	**自動建置 AMI 與容器映像**	EC2 Image Builder、配方、分發、排程
1.4	**各平台部署與儲存模式**	EC2、ECS、EKS、Lambda；EFS、S3、EBS
1.4	**權限、代理程式與疑難排解**	執行個體設定檔、CodeDeploy 代理程式、AppSpec 勾點
1.4	**運用各種部署方法**	可變 vs 不可變、藍綠、金絲雀、線性、滾動